

Waraq — ورق

Complete Codebase Audit & Refactoring Assessment

تدقيق شامل للكود وتقييم إعادة الهيكلة

1. Executive Summary | الملخص التنفيذي

البُعد / Dimension	التقييم / Rating
جودة الكود / Overall Codebase Quality	يحتاج تحسين / Needs Improvement
جودة الهيكلية / Architecture Quality	مقبولة (مرحلة مبكرة) / Acceptable (early-stage)
أبرز نقاط القوة / Biggest Strengths	فصل الكتب عن الإعلانات، مصادقة JWT صحيحة، استخدام جيد لـ Kotlin data classes
أبرز المشاكل / Biggest Problems	مفاتيح سرية مكتوبة بالكود، دوال مكررة بالـ Repository، اختبارات معطلة، رفع ملفات بدون مصادقة، قائمة سوداء للتوكنات في الذاكرة فقط
التقني Overall Technical Debt / الدين	متوسط / Moderate
ضرورة إعادة الهيكلة / Overall Refactoring Necessity	targeted fixes, not a rewrite — Moderate — إصلاحات مستهدفة وليس إعادة كتابة

Overall Assessment: Needs Improvement

Waraq is a marketplace for buying, selling, and exchanging books, targeting **the general public as well as university students** in Jordan. The core architecture (Controller → Service → Repository → DB) is sound and consistently followed. However, there are **critical security issues** (hardcoded JWT secret, database credentials in version control, unauthenticated file uploads), **duplicate query methods** in the repository layer, **stale/broken tests** that reference a previous data model, and several missing features (no update/delete for listings, no pagination metadata). The codebase is small (~38 source files, ~2,200 lines of Kotlin) and the problems are fixable without architectural redesign.

AR: هذا مشروع في مراحله الأولى مبني بـ Spring Boot + Kotlin لمنصة ورق (Waraq) — سوق لبيع وشراء وتبادل الكتب، يستهدف العامة بالإضافة لطلاب الجامعات في الأردن. الهيكلية الأساسية (Controller → Service → Repository → DB) سليمة ومتبعة بشكل ثابت، لكن توجد مشاكل أمنية درجة (مفتاح JWT ثابت بالكود، بيانات قاعدة البيانات محفوظة في نظام التحكم بالإصدارات، رفع ملفات بدون مصادقة)، دوال بحث مكررة في طبقة الـ Repository، اختبارات معطلة تشير لنموذج بيانات قديم،

وعدة ميزات ناقصة. حجم الكود صغير (~38 ملف مصدري، ~2,200 سطر Kotlin) والمشاكل قابلة للإصلاح بدون إعادة تصميم معماري.

2. Project Overview | نظرة عامة على المشروع

What the Project Does | ماذا يفعل المشروع

is a REST API backend for a book marketplace where **the general public and university students** in **EN: Waraq (ورق)** :Jordan can buy, sell, and exchange books. It supports

- User registration & login (JWT-based)
- Creating book listings (academic books tied to university/faculty/major, or general books/novels for anyone)
- Searching/filtering listings by category, university, faculty, major, price, etc
- File upload for book cover images
- Academic hierarchy lookup (universities → faculties → majors)

AR: ورق (Waraq) هو واجهة برمجة تطبيقات (REST API) لسوق كتب حيث يمكن **للعمامة وطلاب الجامعات** في الأردن بيع وشراء وتبادل الكتب. يدعم:

- تسجيل المستخدمين وتسجيل الدخول (مبني على JWT)
- إنشاء إعلانات كتب (كتب أكاديمية مرتبطة بجامعة/كلية/تخصص، أو كتب عامة/روايات لأي شخص)
- البحث والتصفية حسب التصنيف، الجامعة، الكلية، التخصص، السعر، إلخ
- رفع صور أغلفة الكتب
- استعراض التسلسل الأكاديمي (جامعات ← كليات ← تخصصات)

Main Technologies | التقنيات الرئيسية

الغرض / Purpose	الإصدار / Version	التقنية / Technology
اللغة الرئيسية / Primary language	2.3.21	Kotlin
إطار العمل / Application framework	4.1.0	Spring Boot
طبقة الاستمرارية / Primary persistence	(managed)	Spring Data JDBC
المصادقة / Authentication	(managed)	Spring Security
قاعدة البيانات / Database	(Alpine) 15	PostgreSQL
توليد وتحقق JWT token generation/validation / JWT	0.11.5	JJWT
تشفير كلمات المرور / Password hashing	(Spring)	BCrypt
نظام البناء / Build system	wrapper	Gradle (Kotlin DSL)
حاوية PostgreSQL المحلية / Local PostgreSQL container	—	Docker Compose

نقطة الدخول | Entry Point

@EnableJdbcRepositories

 مع WaraqaApplication.kt — Spring Boot main class •

الوحدات الرئيسية | Main Modules

الغرض / Purpose	الحزمة / Package
REST API endpoints / (controllers 5) نقاط API النهائية	controller
Business logic / (services 3) منطق الأعمال	service
Data access / (repositories 4) الوصول للبيانات	repository
Domain entities / (4 ملفات) كيانات النطاق	model
Request/response DTOs / (DTOs 10) كائنات نقل البيانات	dto
Custom exceptions + global handler / (5 ملفات) استثناءات مخصصة + معالج عام	exception
JWT filter, utils, config, blacklist / (5 ملفات) فلتر JWT، أدوات، تكوين، قائمة سوداء	security
Web/static resource config / (ملف واحد) تكوين الويب/الموارد الثابتة	config

3. Architecture Overview | نظرة عامة على الهيكلية

Actual Architecture | الهيكلية الفعلية

EN: The project follows a **standard 3-layer Spring Boot architecture**. AR
المشروع يتبع **هيكلية Spring Boot القياسية ذات 3 طبقات**.

graph TD

Client["العميل / Client (Mobile/Web)"]

subgraph Controllers

AC["AcademicController"]

AUTH["AuthController"]

FU["FileUploadController"]

LC["ListingController"]

UC["UserController"]

end

subgraph Services

AS["AuthService"]

BS["BookService"]

LS["ListingService"]

end

subgraph Repositories

AR["AcademicRepository"]

BR["BookRepository"]

LR["ListingRepository"]

UR["UserRepository"]

end

subgraph Security

JF["JwtAuthenticationFilter"]

JU["JwtUtils"]

SC["SecurityConfig"]

SU["SecurityUtils"]

TBS["TokenBlacklistService"]

end

DB["PostgreSQL"]

Client → JF → Controllers

AUTH → AS → UR

AC → AR

LC → LS → BR

LS → LR

LS → UR

UC → UR

FU → |نظام الملفات / Filesystem| Uploads["uploads/ directory"]

AR → DB

BR → DB

LR → DB

UR → DB

AS → JU

AS → TBS

JF → JU

JF → TBS

Layer Responsibilities | مسؤوليات الطبقات

الطبقة / Layer	المسؤولية / Responsibility	الحالة / Status
Controllers	معالجة HTTP، تحليل الطلبات، تنسيق الاستجابات	نظيفة غالباً مع بعض المشاكل الصغيرة
Services	منطق الأعمال، التحقق، التنسيق	ListingService هو الأساسي؛ BookService غير مستخدم
Repositories	استعلامات SQL، ربط البيانات	خليط من JDBC template و Spring Data CRUD؛ يوجد تكرار
Security	مصادقة CORS، JWT، حماية المسارات	يعمل لكن فيه مفاتيح ثابتة
Models	ربط كيانات JDBC	Kotlin data classes نظيفة
DTOs	عقود API	معقولة مع بعض التداخل

Architectural Observations | ملاحظات معمارية

:EN

Mixed persistence approach: UserRepository extends CrudRepository (auto-generated queries), while BookRepository, ListingRepository, and AcademicRepository use manual NamedParameterJdbcTemplate. Inconsistent but not necessarily wrong — the manual repos need complex queries.

No service layer for some controllers: AcademicController calls AcademicRepository directly. UserController calls UserRepository directly.

No JPA despite the dependency: spring-boot-starter-data-jpa is in build.gradle.kts but the project uses Spring Data JDBC exclusively. JPA is unused.

:AR

1. نهج استمرارية مختلط: UserRepository يستخدم CrudRepository (استعلامات تلقائية)، بينما BookRepository و ListingRepository و AcademicRepository تستخدم NamedParameterJdbcTemplate يدوياً. غير متسق لكن ليس بالضرورة خطأ — ال repositories اليدوية تحتاج استعلامات معقدة.
2. بعض ال Controllers بدون طبقة Service: AcademicController يستدعي AcademicRepository مباشرة. UserController يستدعي UserRepository مباشرة.
3. JPA موجود كاعتماد لكن غير مستخدم: spring-boot-starter-data-jpa موجود بـ build.gradle.kts لكن المشروع يستخدم Spring Data JDBC حصراً.

4. جرد الملفات الكامل | Complete File Inventory

ملفات الإعداد والبناء | Configuration & Build Files

الملف / File	النوع / Type	الغرض / Purpose	المشاكل / Issues	إعادة / Refactoring
build.gradle.kts	بناء / Build	تكوين Gradle والاعتمادات	اعتماد data-jpa غير مستخدم؛ data-jdbc data-jpa معاً	إزالة اعتماد JPA
settings.gradle.kts	بناء / Build	اسم المشروع الجذري	لا توجد مشاكل	لا يحتاج
docker-compose.yml	بنية تحتية / Infra	حاوية PostgreSQL 15	كلمة مرور ثابتة password123	استخدام متغيرات بيئة
application.yaml	إعداد / Config	اتصال قاعدة البيانات، وضع تهيئة SQL	بيانات الاعتماد ثابتة وم محفوظة في VCS؛ غير مضاف لـ .gitignore	يجب إضافته لـ .nore. واستخدام متغيرات
application.yml.example	إعداد / Config	قالب تكوين نموذجي	ينقصه إعداد JWT secret	إضافة placeholder لـ
schema.sql	قاعدة بيانات DB /	مخطط + بيانات أولية (جامعات، كليات، تخصصات)	mode: always يعيد التشغيل كل مرة؛ البيانات الأولية مخلوطة مع المخطط	فصل البيانات الأولية؛ أداة migration
gitignore.	إعداد / Config	قواعد تجاهل Git	ينقصه application.yaml uploads/ ومجلد	إضافة الملفات الحسا
README.md	توثيق / Docs	تعليمات الإعداد	بسيط جداً؛ لا يوجد توثيق API	تحسين التوثيق

Application Entry | نقطة دخول التطبيق

المشاكل / Issues	الغرض / Purpose	الملف / File
No significant issues / لا توجد مشاكل مهمة	Spring Boot main class	WaraqaApplication.kt

المتحكمات | Controllers

الأولوية / Priority	المشاكل / Issues	الغرض / Purpose	الملف / File
P2	معالج استثناءات محلي يكرر العام؛ تنسيق استجابة غير متسق بين register و login	تسجيل، دخول، خروج / Register, Login, Logout	AuthController.kt
P2	imports مكررة؛ نقاط نهائية ناقصة (GET by ID, PUT, DELETE)	إنشاء + بحث إعلانات / Create + search listings	ListingController.kt
—	يستدعي repository مباشرة (بدون service) — مقبول للقراءة فقط	بحث جامعات/كليات / تخصصات / University/faculty/major lookup	AcademicController.kt
P1	URL ثابت localhost:8080 ؛ لا يوجد حد لحجم الملف؛ لا يحتاج مصادقة	رفع صور / Image upload	FileUploadController.kt
P3	يستدعي repository مباشرة؛ ResponseEntity<Any> غير محدد النوع	ملف المستخدم / User profile	UserController.kt

DTOs | كائنات نقل البيانات

الأولوية / Priority	المشاكل / Issues	الغرض / Purpose	الملف / File
—	جيد — annotations تحقق و JSON aliases	مدخلات التسجيل / Registration input	RegisterRequest.kt
P3	لا يوجد @NotBlank على email/password	مدخلات الدخول / Login input	LoginRequest.kt
—	لا توجد مشاكل مهمة	استجابة الدخول مع token / Login response	AuthResponse.kt
—	بدون validation annotations (التحقق يدوي بالـ service) — مقبول	مدخلات إنشاء إعلان / Listing creation input	CreateListingRequest.kt
—	لا توجد مشاكل مهمة	مخرجات الإعلان / Listing output	ListingResponseDto.kt
P2	يُشتبه أنه غير مستخدم — لا يوجد endpoint يستهلكه	مدخلات إنشاء كتاب / Book creation input	CreateBookRequest.kt
P2	يُشتبه أنه غير مستخدم — لا يوجد endpoint يستهلكه	مدخلات تحديث كتاب / Book update input	UpdateBookRequest.kt
P2	يُشتبه أنه غير مستخدم — لم يتم إنشاؤه في أي مكان	مخرجات كتالوج الكتب / Book catalog output	BookResponseDto.kt
P2	يُشتبه أنه غير مستخدم — لا service/controller ينشئه	عرض تفصيلي للكتاب مع بيانات البائع / Detailed book view with seller info	BookDetailDto.kt
—	لا توجد مشاكل مهمة	مخرجات ملف المستخدم / User profile output	UserProfileResponse.kt

النماذج | Models

الأولوية / Priority	المشاكل / Issues	الغرض / Purpose	الملف / File
P3	كلمة المرور مخزنة بالنموذج (تظهر بكل السياقات)	User entity / كيان المستخدم	User.kt
—	نظيف؛ لا توجد مشاكل مهمة	Book / كيان كتالوج الكتب catalog entity	Book.kt
—	column mappings؛ نظيف؛ صحيحة	كيان الإعلان (بيع/تبادل) / Listing entity	Listing.kt
—	data classes بسيطة — جيدة للقراءة فقط	جامعة، كلية، تخصص / University, Faculty, Major	AcademicLookup.kt

المستودعات | Repositories

الأولوية / Priority	المشاكل / Issues	الغرض / Purpose	الملف / File
—	يستخدم CrudRepository بينما الباقي يستخدم JDBC template — غير متسق لكن يعمل	CRUD للمستخدمين (Spring Data)	UserRepository.kt
P2	findAll() يستخدم RowMapper مختلف ينقصه حقل edition : رسالة خطأ بالعربي بـ save()	CRUD للكاتب (JDBC template)	BookRepository.kt
P1	g findListings() findListingsWithFilters() دوال مكررة؛ findListingsWithFilters يستخدم RowMapper مختلف ينقصه viewsCount , imageUrl , publishedAt , savesCount parameter subtype لا يُستخدم بالاستعلام	CRUD + بحث لإعلانات (JDBC template)	ListingRepository.kt
—	نظيف؛ استعلامات مُعلّمة صحيحة	بحث جامعات/كليات/تخصصات (JDBC template)	AcademicRepository.kt

الخدمات | Services

الأولوية / Priority	المشاكل / Issues	الغرض / Purpose	الملف / File
P3	شُعْزَف داخل <code>RegistrationException</code> ملف الـ service بدل حزمة exception: <code>cClearContext()</code> بلا فائدة	تسجيل، دخول، خروج / Registration, login, logout	AuthService.kt
P2	غير مستخدم — 3 دوال، لا يتم استدعاء أي منها من أي controller	عمليات كتالوج الكتب Book catalog / operations	BookService.kt
P1	مشكلة N+1 بـ <code>getListings()</code> ؛ يستخدم <code>SecurityContextHolder</code> مباشرة بدل <code>createListing</code> دالة <code>SecurityUtils</code> كبيرة (~107 أسطر)	إنشاء + بحث إعلانات / Create + search listings	ListingService.kt

Security | الأمان

الأولوية / Priority	المشاكل / Issues	الغرض / Purpose	الملف / File
P1	CORS يسمح فقط بـ localhost! endpoint الرفع عام (بدون مصادقة)	سلسلة الفلاتر، CORS، قواعد المسارات	SecurityConfig.kt
P0	مفتاح JWT ثابت بالكود المصدري	توليد/تحقق التوكن / Token generation/validation	JwtUtils.kt
—	تنفيذ نظيف	استخلاص JWT من الـ header	JwtAuthenticationFilter.kt
P3	<code>getCurrentUserId()</code> يعمل استعلام DB كل مرة — غير مُخزَّن مؤقتاً	الحصول على email/ID المستخدم الحالي	SecurityUtils.kt
P1	القائمة تُفقد عند إعادة التشغيل؛ تنمو بلا حدود (تسرب ذاكرة)	قائمة سوداء بالذاكرة / In-memory blacklist	TokenBlacklistService.kt

الاستثناءات | Exceptions

الأولوية / Priority	المشاكل / Issues	الغرض / Purpose	الملف / File
P2	رسالة ثابتة "Book not found"؛ لا يعالج BadCredentialsException	ربط استثناءات بـ HTTP responses	GlobalExceptionHandler.kt
P3	يُشتبه أنه غير مستخدم — لا يُرمى في أي مكان	Bad Request 400	BadRequestException.kt
—	مستخدم بـ ListingService.createListing()	Forbidden 403	ForbiddenException.kt
—	مستخدم بـ BookService	Not Found 404	NotFoundException.kt
—	مستخدم بـ ListingService	خريطة أخطاء التحقق / Validation errors map	ValidationException.kt

الإعدادات | Config

المشاكل / Issues	الغرض / Purpose	الملف / File
لا توجد مشاكل مهمة	تقديم الملفات المرفوعة كموارد ثابتة	WebConfig.kt

الاختبارات | Tests

الأولوية / Priority	المشاكل / Issues	الغرض / Purpose	الملف / File
—	يحتاج PostgreSQL يكون شغال	اختبار تحميل السياق Context load / smoke test	WaraqaApplicationTests.kt
—	يحتاج PostgreSQL؛ بنية جيدة	اختبار تكاملي لتدفق المصادقة + القائمة السوداء	SecurityTest.kt
P1	معطل — يشير لخصائص Book.price Book.publishedAt Book.coverImage Book.publisherId و دالة findByPublisherId() التي لا توجد بالنموذج الحالي	اختبار حفظ/استرجاع كتاب	BookRepositoryTest.kt

.5 Main Application Flows | تدفقات التطبيق الرئيسية

Flow 1: User Registration | تسجيل المستخدم

Client POST /api/auth/register

- JwtAuthenticationFilter (عام endpoint - يمر)
- SecurityConfig (permitAll لـ /api/auth/**)
- AuthController.register()
 - @Valid RegisterRequest (الاسم، الإيميل، كلمة المرور، الهاتف) يتحقق من
 - AuthService.register()
 - UserRepository.existsByEmail() - فحص التكرار
 - UserRepository.existsByPhoneNumber() - فحص التكرار
 - PasswordEncoder.encode() - تشفير كلمة المرور
 - UserRepository.save() - حفظ المستخدم
- 201 يُرجع {success: true, message: "Registration successful"}

Flow 2: User Login | تسجيل الدخول

Client POST /api/auth/login

- AuthController.login()
 - AuthService.login()
 - UserRepository.findByEmail()
 - PasswordEncoder.matches() - التحقق من كلمة المرور
 - JwtUtils.generateToken() - إنشاء JWT
- 200 يُرجع AuthResponse {token, userId, name, email, phoneNumber}

Flow 3: Create Listing (Authenticated) | إنشاء إعلان (يحتاج مصادقة)

Client POST /listings (مع Authorization: Bearer <token>)

- JwtAuthenticationFilter
- JwtUtils.validateToken()
- TokenBlacklistService.isBlacklisted()
- SecurityContext يُعَيَّن مع email كـ principal
- ListingController.createListing()
- ListingService.createListing()
 - SecurityContextHolder → المستخدم email يحصل على
 - UserRepository.findById() → المستخدم ID يحصل على
 - تحقق يدوي (عنوان، صورة، تصنيف، سعر، حالة، إلخ)
 - BookRepository.findByIdAndAuthor() - إنشاء جديد - عادة استخدام كتاب موجود أو إنشاء جديد
 - BookRepository.save() - إذا كتاب جديد
 - ListingRepository.save() - إنشاء إعلان مربوط بالكتاب
- ListingResponseDto يُرجع 201

Flow 4: Search Listings (Public) | بحث الإعلانات (عام)

Client GET /listings?search=X&category=Y&university_id=Z&sort=newest&page=0&limit=8

- ListingController.getListings()
- ListingService.getListings()
 - ListingRepository.findListingsWithFilters() - استعلام SQL JOIN
 - ⚠️ مشكلة N+1: BookRepository.findById() لكل إعلان
 - List<ListingResponseDto> يُحوّل لـ
- List<ListingResponseDto> يُرجع 200

Flow 5: Academic Lookup (Public) | البحث الأكاديمي (عام)

Client GET /api/universities

- AcademicController.getUniversities()
- AcademicRepository.findAllUniversities()
- 200 يُرجع List<University>

Client GET /api/faculties?university_id=1

- AcademicController.getFaculties()
- AcademicRepository.findFaculties(universityId=1)
- 200 يُرجع List<Faculty>

Flow 6: File Upload (Public — NO AUTH) | رفع ملف (عام — بدون مصادقة)

Client POST /api/upload (multipart file)

- FileUploadController.uploadFile()
 - image يتحقق أنه غير فارغ + نوع
 - UUID يُولد اسم
 - uploads/ يحفظ بنظام الملفات
- 200 يُرجع {url: "http://localhost:8080/uploads/<uuid>.ext"}

Findings by Category | النتائج حسب الفئة

Architecture 6.1 | الهيكلية

الخطورة / Severity	النتيجة / Finding	الموقع / Location	التوصية / Recommendation
متوسط / Medium	أنماط استمرارية مختلفة (CrudRepository مع JdbcTemplate)	UserRepository مقابل الباقي	مقبول — يُوثق النمط
منخفض / Low	بعض الـ controllers تتجاوز طبقة الـ service	AcademicController , UserController	مقبول لعمليات القراءة البسيطة
متوسط / Medium	اعتماد JPA غير مستخدم	build.gradle.kts:23	إزالة spring-boot-starter-data-jpa
منخفض / Low	RegistrationException مُعرّف بملف AuthService.kt بدل حزمة exception	AuthService.kt:15-18	نقله لحزمة exception

Business Logic 6.2 | منطق الأعمال

الموقع / Location	الخطورة / Severity	النتيجة / Finding	التوصية / Recommendation
ListingService.kt:156	High / عالي	مشكلة N+1 بـ <code>getListings()</code> — كل إعلان يستعلم عن كتابه بشكل فردي	إنشاء استعلام بيانات الإعلان و
ListingService.kt:24-131	Medium / متوسط	<code>createListing()</code> دالة 107 سطر تخط التحقق مع الاستمرارية	استخلاص التحدث منفصلة
ListingService.kt:26	Medium / متوسط	<code>ListingService</code> يستخدم <code>SecurityContextHolder</code> مباشرة بدل <code>SecurityUtils</code>	حقن واستخدام <code>SecurityUtils</code>
BookService.kt	Medium / متوسط	<code>BookService</code> موجود لكن لا يُستدعى من أي controller	ربطه بـ controller
AuthService.kt:69	Low / منخفض	<code>AuthService.logout()</code> يستدعي <code>SecurityContextHolder.clearContext()</code> بلا فائدة في JWT stateless	إزالة الاستدعاء

API 6.3 | واجهة البرمجة

Recommendation	الموقع / Location	النتيجة / Finding	الخطورة / Severity
توحيد لـ listings	ListingController.kt:15	بادئات مسار API غير متسقة: /api , /api/auth (/api بدون) /listings	متوسط / Medium
تعريف غلاف	عددة controllers	تنسيقات استجابة غير متسقة: register يُرجع {success, message} login يُرجع AuthResponse مباشرة	متوسط / Medium
نقله لـ ExceptionHandler	AuthController.kt:52-76	لديه AuthController محلي يتنافس مع GlobalExceptionHandler	متوسط / Medium
إضافتها عند	ListingController	نقاط نهائية ناقصة: لا يوجد GET /listings/{id} , لا PUT/DELETE للإعلانات	منخفض / Low
إضافة غلاف	ListingController.getListings()	لا يوجد معلومات صفحات (عدد كلي، إجمالي الصفحات)	منخفض / Low

Database 6.4 | قاعدة البيانات

الخطورة / Severity	النتيجة / Finding	الموقع / Location	التوصية / Recommendation
عالي / High	دوال استعلام مكررة: g findListings() findListingsWithFilters() بـ ListingRepository تعمل نفس الشيء تقريباً	ListingRepository.kt:95-211	إزالة findListings() findListingsWithFilters()
عالي / High	findListingsWithFilters() يستخدم RowMapper مُضيق ينقصه حقول imageUrl , saveCount , viewsCount publishedAt	ListingRepository.kt:195-210	استخدام الـ RowMapper مستوى الكلاس
متوسط / Medium	findListingsWithFilters() يقبل parameter subType لكن لا يستخدمه بالـ SQL	ListingRepository.kt:149	إما تطبيق الفلتر أو إزالة الـ parameter
متوسط / Medium	BookRepository.findAll() يستخدم RowMapper مُضيق ينقصه حقل edition	BookRepository.kt:71-82	استخدام الـ RowMapper مستوى الكلاس
منخفض / Low	لا يوجد index على listings.book_id رغم أنه FK مستخدم بالـ JOINS	schema.sql	إضافة CREATE INDEX IF NOT EXISTS listings_book_id ON listings(book_id)

Security 6.5 | الأمان

Location	النتيجة / Finding	الخطورة / Severity
tils.kt:14	مفتاح JWT ثابت بالكود: "waraqa_dev_secret_key_must_be_at_least_32_bytes_long_123456"	Critical / حرج
n.yaml:5	بيانات اعتماد قاعدة البيانات محفوظة بـ VCS: password123 application.yaml غير مضاف لـ gitignore	Critical / حرج
fig.kt:63	endpoint رفع الملفات لا يحتاج مصادقة — أي شخص يقدر يرفع ملفات على السيرفر	High / عالي
vice.kt:8	القائمة السوداء بالذاكرة — كل التوكنات المحظورة تُفقد عند إعادة تشغيل السيرفر	High / عالي
vice.kt:8	القائمة السوداء تنمو بلا حدود — لا تنظيف TTL، تسرب ذاكرة محتمل	High / عالي
ller.kt:30	لا يوجد قائمة بيضاء لامتدادات الملفات عند الرفع — يقبل أي image/* MIME type يمكن تزويره	Medium / متوسط
ttroller.kt	لا يوجد حد لحجم الملف عند الرفع	Medium / متوسط
ller.kt:42	URL رفع ثابت http://localhost:8080 — لن يعمل بالإنتاج	Medium / متوسط

Code Quality 6.6 | جودة الكود

الموقع / Location	النتيجة / Finding	الخطورة / Severity
BookRepository.kt:40, ListingRepository.kt:76	رسائل خطأ عربية مخلوطة مع إنجليزية بالـ repositories	متوسط / Medium
AuthController.kt:76	تعليق متبقي // ← Added missing closing brace here!	منخفض / Low
ListingRepository.kt:160	مكتوب بالاسم MapSqlParameterSource الكامل بدل import	منخفض / Low

6.7 | الاختبارات Testing

التوصية / Recommendation	النتيجة / Finding	الخطورة / Severity
إعادة كتابة الاختبار للنموذج الحالي أو حذفه	BookRepositoryTest معطل — يشير لخصائص من نموذج Book قديم (price , publishedAt , publisherId , coverImage) ودالة findByIdByPublisherId() غير موجودة	عالي / High
إضافة اختبارات وحدة لمنطق التحقق	لا يوجد اختبارات لـ ListingService (أعقد service)	متوسط / Medium
إضافة اختبارات تكاملية لاستعلامات البحث	لا يوجد اختبارات لدوال استعلام ListingRepository	متوسط / Medium
إضافة Testcontainers لـ CI	كل الاختبارات تحتاج PostgreSQL شغال (لا testcontainers أو H2)	منخفض / Low

6.8 | Dependencies & Configuration الاعتمادات والتكوين

الخطورة / Severity	النتيجة / Finding	التوصية / Recommendation
متوسط / Medium	اعتماد غير مستخدم: spring-boot-starter-data-jpa — المشروع يستخدم JDBC مش JPA. يسحب Hibernate وكل اعتماداته بدون فائدة	إزالته
منخفض / Low	مجلد uploads/ (30 صورة، ~8MB) محفوظ بـ git	إضافته لـ .gitignore

7. What Is Already Good | ما هو جيد ولا يحتاج تغيير

EN: The following should NOT be unnecessarily refactored. **AR [TIP!]** التالي لا يجب إعادة هيكلة بدون سبب مقنع.

1. **فصل الكتب عن الإعلانات / Book/Listing separation** — قرار تصميمي جيد. جدول `books` لبيانات الكتاب الثابتة، وجدول `listings` لعروض البيع/التبادل. يمنع تكرار بيانات الكتاب عبر الإعلانات.

2. **Kotlin data classes للنماذج والـ DTOs** — نظيفة، اصطلاحية Kotlin. غير قابلة للتعديل (immutable)، مختصرة.

3. **تحقق RegisterRequest** — استخدام جيد لـ `@Pattern`، `@Email`، `@NotBlank` مع رسائل واضحة و `@JsonAlias` لمرونة الإدخال.

4. **تدقق مصادقة JWT** — `SecurityConfig` → `JwtUtils` → `JwtAuthenticationFilter` — هيكلي جيداً، يتبع أنماط Spring Security بشكل صحيح.

5. **AcademicRepository** — نظيف، منظم، استعلامات مُعلّمة صحيحة RowMappers متسقة.

6. **SecurityTest** — اختبار تكاملي جيد البنية يغطي endpoints العامة، الوصول المصادق، وتعطيل التوكن بعد الخروج.

7. **تصميم المخطط (schema)** — استخدام صحيح للمفاتيح الأجنبية، القيود (`UNIQUE` ، `NOT NULL`)، الفهارس، و `ON CONFLICT` لبيانات أولية متكررة الإدخال.

8. **منطق التحقق بـ ListingService.createListing()** — تحقق شامل من متطلبات كل تصنيف (أكاديمي مقابل عام)، قيود نوع الإعلان، وقواعد السعر.

9. **WebConfig static resource handler** — تنفيذ بسيط وصحيح لتقديم الملفات المرفوعة.

10. **تفويض مسارات SecurityConfig** — فصل واضح بين endpoints العامة والمحمية.

8. Refactoring Candidates | مرشحات إعادة الهيكلة

P0 — حرج / Critical (يجب الإصلاح فوراً / Must Fix Immediately)

#	File(s) / الملفات	Problem / المشكلة	Proposed Change / التغيير المقترح	Risk / المخاطر	Severity / شدة
1	JwtUtils.kt	مفتاح JWT ثابت بالكود / Hardcoded JWT secret	نقله لـ application.yaml كـ jwt.secret ؛ حقنه بـ @Value	منخفض — التوكنات الحالية ستبطل إذا تغير المفتاح	منخفض
2	application.yaml, .gitignore	بيانات اعتماد DB محفوظة بـ VCS	إضافة application.yaml كـ uploads/ ؛ .gitignore ؛ استخدام application.yaml.example كقالب	منخفض — المطورون يحتاجون إعادة إنشاء الإعداد المحلي	منخفض

P1 — عالي / High

#	الملفات / File(s)	المشكلة / Problem	التغيير المقترح / Proposed Change
3	ListingRepository.kt	دوال findListings() / findListingsWithFilters() مكررة؛ RowMapper ناقص	إزالة findListings() ؛ إصلاح findListingsWithFilters() للاستخدام rowMapper على مستوى الكلاس
4	ListingService.kt	مشكلة N+1 بـ getListings() — bookRepository.findById() لكل إعلان	إنشاء دالة repository تعمل JOIN بين listings+books؛ أو جلب الكتب دفعة واحدة
5	TokenBlacklistService.kt	القائمة السوداء بالذاكرة تُفقد عند الإعادة؛ لا تنظيف TTL	كحد أدنى: إضافة تنظيف مجدول. المثالي: حفظ بـ DB أو Redis
6	SecurityConfig.kt	endpoint الرفع يسمح برفع ملفات بدون مصادقة	نقل /api/upload للقواعد المصادق عليها
7	BookRepositoryTest.kt	الاختبار يشير لخصائص من نموذج Book قديم — لن يعمل compile	إعادة كتابة الاختبار للنموذج الحالي (بدون price, publisherId)
8	FileUploadController.kt	URL ثابت http://localhost:8080	استخدام @Value أو ServletUriComponentsBuilder

P2 — متوسط / Medium

#	الملفات / File(s)	المشكلة / Problem	التغيير المقترح / Proposed Change
9	ListingController.kt	بادئة API غير متسقة (/listings مقابل (/api/ ...	تغيير لـ /api/listings : تحديث SecurityConfig
10	build.gradle.kts	اعتماد data-jpa غير مستخدم	إزالة spring-boot-starter-data-jpa
11	BookRepository.kt	findAll() RowMapper مُضْمَن ينقصه edition	استخدام rowMapper على مستوى الكلاس
12	AuthController.kt	@ExceptionHandler محلي يكرر المعالج العام	نقل معالجة MethodArgumentNotValidException لـ GlobalExceptionHandler
13	:DTOs CreateBookRequest UpdateBookRequest BookResponseDto BookDetailDto	يُشتبه أنها غير مستخدمة	التحقق؛ الإزالة إذا كانت كود ميت فعلياً

P3 — منخفض / Low

#	الملفات / File(s)	المشكلة / Problem	التغيير المقترح / Proposed Change
14	AuthService.kt	RegistrationException معرّف بملف الـ service	نقله لحزمة exception
15	AuthService.kt	clearContext() بـ logout() فائدة	إزالة الاستدعاء
16	repositories	رسائل خطأ عربية ("فشل استرجاع الـ ID")	توحيد اللغة
17	AuthController.kt:76	تعليق تصحيح أخطاء: // ← Added missing closing brace here!	إزالته
18	LoginRequest.kt	لا يوجد @NotBlank على email/password	إضافة تحقق

9. Quick Wins | مكاسب سريعة

AR **EN:** Low-risk, low-effort changes with high benefit. تغييرات منخفضة المخاطر والجهد مع فائدة عالية.

#	Change / التغيير	Effort / الجهد	Benefit / الفائدة
1	إضافة application.yaml و uploads/ إلى .gitignore	2 دقيقة	منع تسريب بيانات الاعتماد
2	إزالة findListings() من ListingRepository (مؤكد غير مستخدم)	5 دقائق	إزالة الارتباك والكود الميت
3	إصلاح findListingsWithFilters() لاستخدام rowMapper على مستوى الكلاس	5 دقائق	إصلاح الحقول المفقودة بنتائج البحث
4	إصلاح BookRepository.findAll() لاستخدام rowMapper على مستوى الكلاس	2 دقيقة	إصلاح حقول edition المفقود
5	إزالة تعليق التصحيح بـ AuthController	1 دقيقة	نظافة الكود
6	إزالة clearContext() من AuthService.logout()	1 دقيقة	إزالة كود مُطل
7	حذف أو إعادة كتابة BookRepositoryTest	15 دقيقة	إصلاح الاختبارات المعطلة
8	إزالة اعتماد JPA غير المستخدم	2 دقيقة	تقليل حجم البناء ووقت البدء

10. Major Refactorings | إعادة هيكلة رئيسية

Major Refactoring 1: Fix N+1 Query | إصلاح مشكلة N+1

- **الملفات / Files:** ListingService.kt , ListingRepository.kt
- **المشكلة / Problem:** `getListings()` يستعلم عن كل كتاب بشكل فردي داخل حلقة
- **التغيير / Change:** إنشاء استعلام JOIN يُرجع بيانات الإعلان والكتاب معاً، أو جلب كل الكتب دفعة واحدة بالـ IDs
- **الأثر / Impact:** تحسين أداء البحث عن الإعلانات (الـ endpoint الأكثر استخداماً)
- **التعقيد / Complexity:** متوسط / Medium

Major Refactoring 2: Externalize All Secrets | إخراج كل المفاتيح السرية

- **الملفات / Files:** FileUploadController.kt , docker-compose.yml , application.yml , JwtUtils.kt
- **المشكلة / Problem:** المفاتيح والقيم الخاصة بالبيئة ثابتة في كل مكان
- **التغيير / Change:** نقل JWT secret، بيانات DB، base URL للتكوين مع تجاوزات متغيرات البيئة
- **الأثر / Impact:** أمان + جاهزية للإنتاج
- **التعقيد / Complexity:** صغير-متوسط / Small-Medium

Major Refactoring 3: Standardize API Response Format | توحيد تنسيق استجابة API

- **الملفات / Files:** كل الـ controllers
- **المشكلة / Problem:** تنسيقات استجابة غير متسقة
- **التغيير / Change:** إنشاء غلاف `ApiResponse<T>` موحد
- **الأثر / Impact:** عقد API أفضل؛ تطوير عميل أسهل
- **التعقيد / Complexity:** متوسط (يمس كل الـ controllers)

11. Technical Debt | الدين التقني

Critical Technical Debt | دين تقني حرج

Debt / الدين	Location / الموقع	Impact / الأثر	Address Now / يُعالج الآن؟
مفتاح JWT ثابت / Hardcoded JWT secret	JwtUtils.kt	أي شخص لديه وصول للكود يقدر يزور توكنات	نعم / Yes
بيانات اعتماد بـ VCS / Credentials in VCS	application.yaml	تعرض بيانات قاعدة البيانات	نعم / Yes

Important Technical Debt | دين تقني مهم

Debt / الدين	Location / الموقع	Impact / الأثر	Address Now / يُعالج الآن؟
قائمة سوداء بالذاكرة / In- memory token blacklist	TokenBlacklistService.kt	الخروج لا يبقى بعد إعادة التشغيل؛ تسرب ذاكرة	قريباً / Soon
مشكلة N+1 / N+1 query	ListingService.kt	الأداء يتدهور مع زيادة الإعلانات	قريباً / Soon
ملف اختبار معطل / Broken test file	BookRepositoryTest.kt	إحساس زائف بتغطية الاختبارات	قريباً / Soon
دوال repository مكررة / Duplicate repo methods	ListingRepository.kt	ارتباك بالصيانة؛ سلوك غير متسق	قريباً / Soon

Acceptable Technical Debt | دين تقني مقبول

الدين / Debt	الموقع / Location	يُعالج لاحقاً / Address Later
لا معلومات صفحات / No pagination metadata	ListingController	عند بناء واجهة الصفحات
أنماط استمرارية مختلطة / Mixed persistence	Repository layer	فقط إذا أصبح مربكاً
CORS localhost-only / CORS فقط	SecurityConfig	عند النشر للإنتاج

Cosmetic Technical Debt | دين تقني تجميلي

الدين / Debt	الموقع / Location
رسائل خطأ عربية/إنجليزية مخلوطة / Mixed language error messages	Repositories, BookService
تعليقات تصحيح أخطاء / Debug comments	AuthController
أسماء كلاسات كاملة بدل imports / Fully qualified class names	ListingRepository, AuthService

12. Dead / Unused Code | كود ميت / غير مستخدم

CONFIRMED UNUSED | مؤكد غير مستخدم

العنصر / Item	الموقع / Location	الدليل / Evidence
دالة <code>findListings()</code>	<code>ListingRepository.kt:95-142</code>	<code>ListingService.getListings()</code> يستدعي <code>findListingsWithFilters()</code> بدلاً منها؛ البحث يؤكد عدم وجود مستدعي آخر
<code>BookService</code> (الكلاس بالكامل)	<code>BookService.kt</code>	لا <code>controller</code> أو <code>service</code> يستدعي أي من دواله
<code>BadRequestException</code>	<code>BadRequestException.kt</code>	لا يُرمى في أي مكان بالكود

SUSPECTED UNUSED | يُشتبه أنه غير مستخدم (ربما مُجهّز للمستقبل)

العنصر / Item	الموقع / Location	الدليل / Evidence
DTO <code>CreateBookRequest</code>	<code>CreateBookRequest.kt</code>	لا controller أو service يشير إليه
DTO <code>UpdateBookRequest</code>	<code>UpdateBookRequest.kt</code>	لا controller أو service يشير إليه
DTO <code>BookResponseDto</code>	<code>BookResponseDto.kt</code>	لم يتم إنشاؤه في أي مكان
DTOs <code>SellerDto</code> + <code>BookDetailDto</code>	<code>BookDetailDto.kt</code>	لم يتم إنشاؤه في أي مكان
<code>AcademicRepository.findUniversityById()</code> <code>findMajorById()</code> , <code>findFacultyById()</code>	<code>AcademicRepository.kt:63-77</code>	لا مستدعي — على الأرجح مُجهز للمستقبل

STALE CODE | كود قديم

العنصر / Item	الموقع / Location	الدليل / Evidence
<code>BookRepositoryTest</code>	<code>BookRepositoryTest.kt</code>	يشير لـ <code>Book.price</code> , <code>Book.publishedAt</code> , <code>Book.publisherId</code> , <code>Book.coverImage</code> — لا يوجد <code>bookRepository.findByPublisherId()</code> أي منها بالنموذج الحالي. هذا الاختبار كُتب لنسخة سابقة قبل فصل الكتب عن الإعلانات.

13. Missing Tests | اختبارات مفقودة

الملف / File	الكلاس/الدالة / Class/Method	السيناريو / Scenario	لماذا / Why
ListingService.kt	createListing()	التحقق يرفض عنوان مفقود / Validation rejects missing title	العملية التجارية الأساسية
ListingService.kt	createListing()	إعلان أكاديمي يحتاج جامعة/كلية/تخصص Academic requires / uni/fac/maj	تحقق خاص بالتصنيف
ListingService.kt	createListing()	يُعيد استخدام كتاب موجود عند تطابق العنوان+المؤلف / Reuses existing book	منطق إزالة التكرار
ListingService.kt	getListings()	يُرجع إعلانات مُصفاة حسب التصنيف والجامعة / Filtered results	وظيفة البحث الأساسية
ListingRepository.kt	findListingsWithFilters()	يُطبق فلاتر البحث والتصنيف والجامعة بشكل صحيح	صحة استعلامات SQL
ListingRepository.kt	save()	يحفظ ويسترجع إعلان بكل الحقول	صحة الاستمرارية
AuthService.kt	register()	يرفض إيميل مكرر / Rejects duplicate email	قاعدة عمل
AuthService.kt	login()	يُرجع token لبيانات صحيحة / Returns token for valid credentials	تدفق المصادقة الأساسي

الملف / File	الكلاس/الدالة / Class/Method	السيناريو / Scenario	لماذا / Why
FileUploadController.kt	uploadFile()	يرفض ملفات غير صور Rejects non-image / files	تحقق أمني
TokenBlacklistService.kt	isBlacklisted()	يُرجع true لتوكن محظور / Returns true for blacklisted token	صحة المصادقة

14. Refactoring Roadmap | خارطة طريق إعادة الهيكلة

Step 1: Fix Security Vulnerabilities (P0) | إصلاح الثغرات الأمنية

الملفات / Files: JwtUtils.kt , application.yaml , .gitignore

- إخراج JWT secret للتكوين / Externalize JWT secret to config
- إضافة application.yaml و uploads/ ل .gitignore
- لماذا أولاً / Why first: ثغرات أمنية يمكن استغلالها فوراً
- الاعتمادات / Dependencies: لا شيء
- المخاطر / Risk: التوكنات الحالية سَتُبطل إذا تغير المفتاح
- الاختبارات بعدها / Tests after: التحقق أن login/auth لا يزال يعمل

Step 2: Fix Broken Tests & Dead Repository Code (P1) | إصلاح الاختبارات المعطلة

والكود الميت

الملفات / Files: BookRepositoryTest.kt , ListingRepository.kt

- إعادة كتابة BookRepositoryTest للنموذج الحالي
- إزالة دالة findListings() المكررة
- إصلاح findListingsWithFilters() لاستخدام rowMapper على مستوى الكلاس
- لماذا ثانياً / Why second: نحتاج اختبارات شغالة قبل أي تغييرات إضافية
- الاعتمادات / Dependencies: لا شيء
- المخاطر / Risk: لا شيء — findListings() غير مستخدمة
- الاختبارات بعدها / Tests after: تشغيل كل الاختبارات؛ التحقق أن بحث الإعلانات يُرجع كل الحقول

Step 3: Fix N+1 Query (P1) | إصلاح مشكلة N+1

الملفات / Files: ListingRepository.kt , ListingService.kt

- إنشاء استعلام JOIN يُرجع بيانات الإعلان والكتاب معاً
- تحديث getListings() لاستخدام الاستعلام الجديد

- لماذا ثالثاً / Why third: الأداء خرج لـ endpoint البحث
- الاعتمادات / Dependencies: الخطوة 2 (repository مُصلح)
- المخاطر / Risk: متوسط — يحتاج اختبار شامل للاستعلام
- الاختبارات بعدها / Tests after: اختبار تكاملي لبحث الإعلانات مع إعلانات متعددة

Step 4: Secure File Upload (P1) | تأمين رفع الملفات

الملفات / Files: application.yaml , FileUploadController.kt , SecurityConfig.kt

- طلب مصادقة لـ /api/upload
- إضافة حدود حجم الملف
- استبدال URL الثابت localhost:8080
- لماذا رابعاً / Why fourth: يمنع إساءة استخدام endpoint الرفع
- الاعتمادات / Dependencies: لا شيء
- المخاطر / Risk: منخفض
- الاختبارات بعدها / Tests after: التحقق أن الرفع المصادق يعمل؛ غير المصادق يُرفض

Step 5: Clean Up Dead Code (P2) | تنظيف الكود الميت

الملفات / Files: DTOs , BookService.kt , BadRequestException.kt

- إزالة الكود المؤكد غير مستخدم
- اتخاذ قرار بشأن DTOs المشتبه بها (الإبقاء لميزات مخططة أو الإزالة)
- لماذا خامساً / Why fifth: يقلل الارتباك؛ كود أنظف
- الاعتمادات / Dependencies: لا شيء
- المخاطر / Risk: منخفض
- الاختبارات بعدها / Tests after: التحقق أن البناء لا يزال يعمل compile

Step 6: Standardize API (P2) | توحيد الـ API

الملفات / Files: GlobalExceptionHandler.kt , AuthController.kt , ListingController.kt

- توحيد بادئة المسار لـ /api/...
- نقل معالج الاستثناءات المحلي للعام

- توحيد تنسيق الاستجابة
- لماذا سادساً / Why sixth: يحسن تناسق API لتطوير العميل
- الاعتمادات / Dependencies: الخطوات 1-4 (الوظائف الأساسية يجب أن تكون مستقرة)
- المخاطر / Risk: متوسط — العملاء يحتاجون تحديث الـ endpoints
- الاختبارات بعدها / Tests after: SecurityTest، اختبار API يدوي

Step 7: Add Missing Tests (P2) | إضافة الاختبارات المفقودة

الملفات / Files: ملفات اختبار جديدة

- إضافة اختبارات لـ ListingService
- إضافة اختبارات تكاملية لـ ListingRepository
- إضافة اختبارات لـ FileUploadController
- لماذا أخيراً / Why last: الآن الكود نظيف؛ الاختبارات تتحقق من الكود المُعاد هيكلته
- الاعتمادات / Dependencies: الخطوات 1-6
- المخاطر / Risk: لا شيء
- الاختبارات بعدها / Tests after: تشغيل كل مجموعة الاختبارات

15. Final Verdict | الحكم النهائي

1. Is the current architecture acceptable? | هل الهيكلية الحالية مقبولة؟

EN: Yes, for an early-stage project. The 3-layer architecture (Controller → Service → Repository) is sound and consistently applied. The code is readable, uses idiomatic Kotlin, and the domain model (separating books from listings — supporting both the general public and university students) is well-structured. This does NOT need an architectural rewrite.

AR: نعم، لمشروع بمراحله الأولى. هيكلية الـ 3 طبقات (Controller → Service → Repository) سليمة ومُتَّبَعَة بشكل ثابت. الكود مقروء، يستخدم Kotlin اصطلاحي، ونموذج النطاق (فصل الكتب عن الإعلانات — مع دعم العامة وطلاب الجامعات) مُصمم جيداً. لا يحتاج إعادة كتابة معمارية.

2. What are the 5 biggest problems? | ما هي أكبر 5 مشاكل؟

1. مفتاح JWT ثابت + بيانات اعتماد DB بـ VCS — ثغرة أمنية
2. مشكلة N+1 يبحث الإعلانات — مشكلة أداء
3. دوال repository مكررة/معطلة — `findListings()` مقابل `findListingsWithFilters()` مع `RowMapper` ناقص
4. قائمة سوداء بالذاكرة — الخروج لا يبقى بعد إعادة التشغيل؛ نمو بلا حدود
5. ملف اختبار معطل — `BookRepositoryTest` يشير لنموذج ميت؛ يعطي ثقة زائفة

3. What must be fixed immediately? | ما يجب إصلاحه فوراً؟

- إخراج JWT secret من الكود المصدري / Externalize JWT secret
- إضافة `application.yaml` لـ `.gitignore`
- إصلاح `RowMapper` الناقص بـ `findListingsWithFilters()` (يُسقط بيانات الصور/المشاهدات بصمت)

4. What should be refactored soon? | ما يجب إعادة هيكلته قريباً؟

- إصلاح مشكلة N+1 بـ `ListingService.getListings()`
- إزالة دالة `findListings()` المكررة
- إعادة كتابة أو حذف `BookRepositoryTest`

- تأمين endpoint رفع الملفات (طلب مصادقة)
- حفظ القائمة السوداء أو إضافة تنظيف TTL

5. What can wait | ما يمكن أن ينتظر؟

- توحيد تنسيق استجابة API
- إضافة معلومات الصفحات (pagination metadata)
- إزالة DTOs المشتبه بعدم استخدامها
- نقل `RegistrationException` لحزمة exception
- إضافة تغطية اختبارات شاملة
- تكوين CORS للإنتاج

6. What should NOT be touched | ما لا يجب لمسه؟

- تحقق `RegisterRequest` — مُنفذ جيداً
- `JwtAuthenticationFilter` — نظيف، صحيح
- `AcademicRepository` — مُوكل جيداً
- `SecurityTest` — اختبار تكاملي جيد
- بيانات `schema.sql` الأولية — استخدام صحيح لـ `ON CONFLICT`
- `WebConfig` — بسيط وصحيح
- نماذج `User` ، `Book` ، `Listing` — Kotlin data classes نظيفة
- الهيكلية العامة ذات الـ 3 طبقات — تعمل

7. Estimated overall refactoring complexity | تعقيد إعادة الهيكلة الإجمالي المُقدّر؟

EN: Small-Medium. The codebase is ~2,200 lines of Kotlin across 38 files. The critical security fixes (Steps 1-2) are trivial. The N+1 fix and API standardization are medium complexity. No architectural changes are needed. A focused developer could complete Steps 1-5 in **1-2 days** and the full roadmap in **3-4 days**.

AR: صغير-متوسط. حجم الكود ~2,200 سطر Kotlin عبر 38 ملف. إصلاحات الأمان الحرجة (الخطوات 1-2) بسيطة. إصلاح N+1 وتوحيد API تعقيدهم متوسط. لا تغييرات معمارية مطلوبة. مطور مُركّز يقدر يُنهي الخطوات 1-5 بـ **2-1 يوم** وخارطة الطريق كاملة بـ **3-4 أيام**.